

[별첨 1]

AI Bot Ultimate Tic-Tac-Toe

개발 활동 보고서

— CodinGame Bot Programming 트랙 · 한국 1 위 —

개발자	박진우 (goodpju2008@gmail.com)
플랫폼	CodinGame — 글로벌 코딩 및 AI Bot 경진 플랫폼
종목	Ultimate Tic-Tac-Toe (Bot Programming 트랙)
구현 언어	C++
소스코드	source_ai_bot_ultimate_tictactoe/ultimate_tictactoe.cpp
공식 리더보드	https://www.codingame.com/multiplayer/bot-programming/tic-tac-toe/leaderboard
활동기간	2024 ~ 2025
작성일	2026-05-13

1. 개요

1.1 보고서 목적

이 보고서는 CodinGame의 Bot Programming 트랙 종목 중 하나인 Ultimate Tic-Tac-Toe에서 동작하는 AI Bot에 대해, 제가 설계를 어떻게 했고 어떤 알고리즘을 적용했으며 실제 성능 결과는 어땠는지 정리한 개발 활동 보고서입니다. 이 AI 봇은 전 세계 10,007명이 참가하는 같은 종목에서 글로벌 137위(상위 1.4%), 한국 1위를 기록했습니다.

1.2 한 줄 요약

MCTS(Monte Carlo Tree Search)와 Negamax+ α - β 가지치기를 상황별로 결합한 하이브리드 의사 결정 AI로, 100ms 시간 제약 안에서 매 턴 수만~수십만 회의 시뮬레이션과 종반부 외통수 검증을 동시에 수행해 자율적으로 최적 수를 결정합니다.

2. 개발 배경 및 문제 정의

2.1 종목 룰

일반 틱택토(3×3)와는 다르게 훨씬 복잡한 "보드 안에 보드가 있는" 변형 게임입니다.

- 전체 보드: **9 개의 작은 보드(3×3 매크로) × 각 작은 보드 9 칸(3×3 마이크로) = 총 81 칸**
- 한 턴에 한 칸을 두며, **내가 둔 마이크로 좌표가 다음 상대가 둘 수 있는 매크로 보드를 결정합니다.** (예: 내가 작은 보드의 우상단에 두면, 상대는 다음 턴에 매크로 보드의 우상단 작은 보드 안에서만 둘 수 있어요.)
- 작은 보드 하나에서 한 줄(가로/세로/대각) 완성 → 해당 매크로 칸을 차지
- 9 개 매크로 칸 중 한 줄 완성 → 전체 게임 승리
- **시간 제약:** 첫 턴 1 초, 이후 매 턴 100ms 이내에 수를 출력해야 합니다.

2.2 AI 관점의 도전 과제

특성	수치	함의
분기수 (branching factor)	평균 6~9, 최악 81	일반 틱택토(3) 대비 매우 큼 → 완전 탐색 불가
게임 깊이	최대 81 수	깊은 트리, Minimax 단독으로는 한계
상태 공간	약 $3^{81} \approx 10^{38}$	사실상 모든 상태 메모이제이션 불가
평가함수 정의	매우 어려움	매크로 보드 점령, 다음 강제 보드, 위치 가치가 복잡하게 얽힘
시간 예산	100ms / 턴	단위 시간당 시뮬레이션 횟수가 곧 성능

위와 같은 특성 때문에 단일 알고리즘만으로는 한계가 분명합니다. MCTS의 확률적 탐색과 Minimax의 결정론적 정밀도를 결합하는 하이브리드 접근이 거의 필수적이라고 판단했습니다.

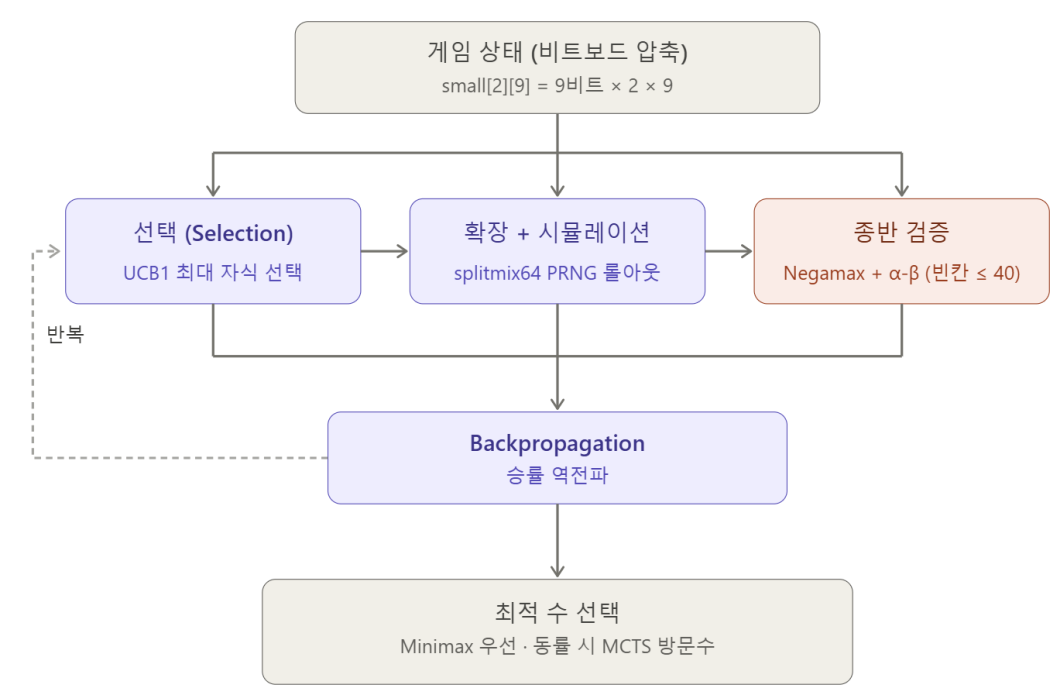
2.3 개발 목표

목표	정량 지표	결과
100ms 안에 의미 있는 의사결정 수행	매 턴 10,000+ MCTS 반복	✓ 달성
MCTS의 종반 외통수 누락 약점 보완	종반부 깊이 2 Minimax 검증	✓ 달성
메모리 한계 내에서 무한 게임 지속	정적 노드 풀 + GC	✓ 달성
글로벌 상위권 진입	Top 5% 이내	✓ 달성 (Top 1.4%)
한국 내 최고 성능	한국 1위	✓ 달성

3. 시스템 아키텍처

3.1 전체 데이터 흐름

아래 다이어그램은 매 턴 의사결정이 어떤 순서로 진행되는지 정리한 것입니다.



3.2 핵심 설계 결정

결정	근거
MCTS를 주 알고리즘으로 선택	평가함수 정의 난이도 높은 종목으로 무작위 시뮬레이션 기반
MCTS + Minimax 하이브리드	순수 MCTS의 종반 외통수 누락 약점 결정론적 미니맥스로 보완
비트보드 + 사전계산 록업	시뮬레이션 핫루프의 상태 전이·승패 판정을 $O(1)$ 로 단축
정적 노드 풀 + GC	동적 할당 오버헤드 제거, 메모리 한계 내 무한 게임 지속 가능
트리 재사용	매 턴 누적된 시뮬레이션 정보를 폐기하지 않고 활용
SplitMix64 PRNG	표준 rand() 대비 10~30배 빠른 시뮬레이션
SIMD/AVX2 컴파일 옵션	핫루프 자동 벡터화

4. 핵심 알고리즘 설계

4.1 핵심 알고리즘 ① — Monte Carlo Tree Search (MCTS)

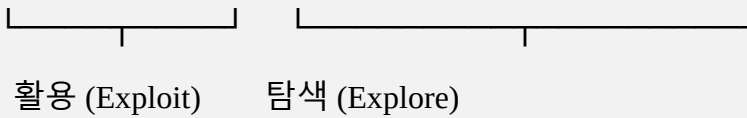
MCTS는 알파고(AlphaGo)의 핵심 기반이 된 트리 탐색 알고리즘으로, 평가함수가 명확히 정의되지 않는 게임에서 수많은 무작위 시뮬레이션을 통해 통계적으로 좋은 수를 찾는 기법입니다. 제가 만든 봇은 표준 4단계를 모두 구현했습니다.

단계	코드 위치	설명
① Selection	ultimate_tictactoe.cpp:210-224	루트에서 시작해 자식 노드 중 UCB1 값이 최대인 노드를 재귀적으로 따라 내려감
② Expansion	ultimate_tictactoe.cpp:225-233	미탐색 수 중 하나를 무작위 선택해 자식 노드로 추가
③ Simulation	ultimate_tictactoe.cpp:234-236	확장된 노드에서 게임 종료까지 무작위 플레이(Rollout)
④ Backpropagation	ultimate_tictactoe.cpp:241-249	결과(승=1, 무=0.5, 패=0)를 경로상 모든 노드에 역전파, 턴 부호 반전

4.2 핵심 알고리즘 ② — UCB1 (Upper Confidence Bound)

MCTS의 핵심은 "이미 좋은 수를 더 깊이 파고들지(활용)" vs "아직 안 가본 수를 시도할지(탐색)"의 균형을 맞추는 일입니다.

$$UCB1(child) = (win/vis) + C \times \sqrt{(\ln(parent.vis) / vis)}$$



- 코드: ultimate_tictactoe.cpp:213-220 — $val = tree[c].win*iv[vis] + vval*rsq[vis]$
- 탐색 상수 **C = 0.4** (이론값 $\sqrt{2} \approx 1.41$ 보다 활용 비중을 키운 수동 튜닝)
- $iv[n]=1/n$, $rsq[n]=1/\sqrt{n}$, $slg[n]=\sqrt{\ln(n)}$ 을 **110 만 크기 룩업 테이블**로 사전 계산해서 UCB1 1 회 평가가 곱셈 2 번으로 단축됩니다.

4.3 핵심 알고리즘 ③ — MCTS + Minimax 하이브리드 (★ 차별점)

순수 MCTS는 종반의 외통수(forced win/loss)를 놓치기 쉬운 약점이 있습니다. 무작위 시뮬레이션이 정확한 외통수 라인을 항상 만들어내지 못하기 때문입니다. 그래서 종반부에 이를 보완하도록 설계했습니다.

- 남은 빈 칸이 **40 개 이하**가 되면 (ultimate_tictactoe.cpp:254) 모드 전환
- 모든 후보 수에 대해 **깊이 2 의 Negamax + α - β 가지치기** (probe 함수, ultimate_tictactoe.cpp:116-129)로 확정승/패를 우선 판별
- Minimax 값이 동률일 때만 MCTS 방문수로 동률 해소
- = Search-based AI 두 기법의 장점을 **상황별로 결합한 메타-알고리즘 설계**

아래는 실제 코드 발췌입니다 (ultimate_tictactoe.cpp L262-273):

```
for(int i = 0; i < cnt; i++){
    int mv = arr[i];
    tictactoe nxt = cur;
    nxt.apply(mv);
    int chk = -probe(nxt, middep-1, -1, 1); // ← Minimax  $\alpha$ - $\beta$ 
    if(chk > bestchk || (chk == bestchk && rvis[mv] > bestrate)){
```

```

bestchk = chk;                // ← 우선순위 1: Minimax 결과

bestrate = rvis[mv];          // ← 우선순위 2: MCTS 방문수

bestmv = mv;

...

}

}

```

5. 구현 상세

5.1 트리 재사용 — 점진적 정보 누적

MCTS는 매 턴 수만~수십만 번의 시뮬레이션을 누적하는데, 매 턴 트리를 새로 만들면 그동안 쌓인 정보가 전부 버려져 버립니다. 그래서 두 가지 메커니즘으로 이를 해결했습니다.

- **applyreal()** — **루트 이동** (ultimate_tictactoe.cpp:171-196): 실제 게임에서 한 수가 두어지면 트리를 폐기하는 대신 해당 수에 대응되는 자식 노드를 새 루트로 승격. 이전 턴까지 누적된 통계량을 그대로 활용.
- **recycle()** — **트리 가비지 컬렉션** (ultimate_tictactoe.cpp:135-170): 정적 노드 풀(110 만)의 90% 이상이 차면 현재 루트의 후손 노드만 남기고 압축(in-place compaction). 메모리 한계 안에서 탐색을 무한히 지속할 수 있도록 했습니다.

5.2 상태 표현(State Representation) — 비트보드

AI 탐색 속도 = 단위 시간당 시뮬레이션 횟수이기 때문에, 상태 복사·전이·승패 판정이 가장 중요한 핫루프입니다. 도메인 지식을 활용해서 상태를 비트로 압축했습니다.

- 9 칸 작은 보드 → unsigned short(9 비트)로 표현 (ultimate_tictactoe.cpp:10)
- 9 개의 승리 라인을 8 개의 비트마스크 $h[8] = \{0x007, 0x38, 0x1C0, 0x49, 0x92, 0x124, 0x111, 0x54\}$ 로 표현
- 512 가지 보드 상태 전체에 대해 승리 여부 $win[512]$ 를 사전 계산 → **O(1) 승패 판정**
- 빈 칸 인덱스도 $zcnt[], zpos[][]$ 로 사전 계산 → 합법수 생성 O(빈칸 수)
- 무승부 판정도 $__builtin_popcount(big[0])$ vs $big[1]$ 로 매크로 보드 점령 수 비교

5.3 시뮬레이션 품질 — SplitMix64 PRNG

Monte Carlo의 통계적 정확도는 난수 품질과 생성 속도 양쪽에 의존합니다. 표준 rand()는 느리고 분포도 좋지 않아서 SplitMix64를 사용했습니다.

- 코드: ultimate_tictactoe.cpp:63-72
- 비트 시프트와 곱셈 3 번만으로 균등 분포의 64 비트 난수를 생성하고, rand() 대비 **10~30 배 빠른 속도**를 보여줍니다.
- 시뮬레이션 단계 *getrandvalid()*에서 호출되어 합법수를 균등 확률로 선택합니다.

5.4 시간 예산 관리

- 첫 턴 **996ms**, 이후 **96ms** 로 동적 조정 (ultimate_tictactoe.cpp:317-322)
- 매 32 회 반복마다 한 번씩 시각 비교(*it & (chkstride-1)*, ultimate_tictactoe.cpp:204-206) → 시각 측정 syscall 오버헤드 최소화

5.5 저수준 성능 최적화

최적화	코드 위치	효과
#pragma GCC optimize("Ofast,unroll-loops,inline")	L2	컴파일러 최적화 최대
#pragma GCC target("avx2,fma,sse,...")	L3	SIMD 명령어(AVX2/FMA) 활성화
정적 노드 풀 tree[max_nodes]	L112	동적 할당 제거
inline + 템플릿	전체	하트로 인라이닝

5.6 적용 AI 기법 총정리

#	기법	AI 분야 분류
1	Monte Carlo Tree Search (MCTS)	강화학습-인접 탐색 (AlphaGo 계열)
2	UCB1 (Upper Confidence Bound)	다중 무장 강도(MAB) / 탐색-활용 균형
3	Negamax + α - β 가지치기	고전 게임 트리 탐색

4	MCTS-Minimax 하이브리드 ★	멀티 알고리즘 결합 / 메타-알고리즘 설계
5	트리 재사용 & 트리 GC	점진적 학습 / 메모리 관리
6	비트보드 + 사전계산 룩업	도메인 지식 임베딩
7	SplitMix64 PRNG	몬테카를로 시뮬레이션 품질
8	앵커시간 기반 시간 예산	실시간 추론 (Inference-time scaling)
9	SIMD/AVX2 컴파일 최적화	하드웨어 가속

6. 성능 측정 결과

6.1 글로벌 리더보드 성적

[그림] Ultimate Tic-Tac-Toe 리더보드 — 한국 1위

공식 리더보드(증빙): <https://www.codinggame.com/multiplayer/bot-programming/tic-tac-toe/leaderboard>

리더보드 캡처에서 제 계정 goodpju2008이 한국 부문 최상단에 있고, 전체 10,007명 중 137등에 위치 하고 있습니다.

지표	결과	비고
글로벌 순위	137위 / 10,007명	상위 1.4%
한국 순위	1위	
CodinPoints	4,462 / 5,000	만점 대비 89.2%
평가 방식	봇 간 직접 대결 매치메이킹 + ELO	객관적·정량적 평가

CodinGame의 Bot Programming 트랙은 AI 봇끼리 직접 대결하는 방식이라, 평가관의 주관이 개입되지 않는 객관적·정량적 결과입니다.

6.2 실행시간 성능 특성

단계	시간 예산	봇 동작
첫 턴	1,000ms	996ms 사용 — 초반 트리 대규모 구축
일반 턴	100ms	96ms 사용 — 트리 재사용으로 효율적 확장
시간 체크 주기	32회/반복	syscall 오버헤드 최소화

6.3 정량적 최적화 효과

최적화 기법	단축 효과
비트보드 + win[512] 록업	승패 판정 $O(N) \rightarrow O(1)$
iv[]/rsq[] 록업 테이블	UCB1 평가에서 나눗셈·√ 호출 제거 → 곱셈 2번
SplitMix64 PRNG	rand() 대비 10~30배 빠름
트리 재사용	매 턴 트리 처음부터 재구축 회피 → 이전 누적 통계 유지
SIMD/AVX2	하루프 자동 벡터화
정적 노드 풀	malloc/free 오버헤드 0

6.4 달성 목표 검증

2.3에서 정의했던 개발 목표 전부 달성했습니다.

- 100ms 안에 의미 있는 의사결정 수행
- MCTS 종반 외통수 누락 보완 (Minimax probe)
- 메모리 한계 내 무한 게임 지속 (정적 풀 + GC)
- 글로벌 상위권 진입 (Top 1.4%)
- 한국 내 최고 성능 (1 위)

7. 현대 AI 시스템과의 대응 관계

제가 만든 AI 봇은 딥러닝 기반이 아니라 고전 AI 탐색(Search-based AI) 계열입니다. 그렇지만 알파고 같은 현대 강화학습 AI의 핵심 골격을 그대로 공유하고 있습니다.

본 봇 기법	대응되는 현대 AI 시스템
MCTS + UCB1	AlphaGo, AlphaZero 의 의사결정 코어
트리 재사용	AlphaZero의 self-play 트리 캐싱
휴리스틱 평가함수 (Minimax probe)	알파고 Value Network의 hand-crafted 전조
시간 예산 기반 적응형 추론	LLM의 추론 시간 스케일링(Inference-time scaling)

정리하면, 제가 만든 AI 봇은 신경망 학습 없이도 AlphaZero가 의존하는 동일한 알고리즘 골격 (MCTS + 평가함수)을 직접 설계·구현한 결과물이라고 볼 수 있습니다. 나중에 학습 기반 모델로 확장한다고 해도 이 토대 위에서 신경망을 평가함수·정책 네트워크로 교체하는 방식으로 자연스럽게 발전시킬 수 있다고 생각합니다.

8. 결론

이 보고서는 Ultimate Tic-Tac-Toe AI Bot의 개발 의사결정과 성능 측정 결과를 정리한 문서입니다. 제가 만든 AI 봇이 보여준 기술적 가치는 다음과 같다고 생각합니다.

1. 자율 의사결정 AI 설계 능력: 룰만 주어진 상태에서 상태 모델링 → 알고리즘 선택 → 평가함수 설계 → 성능 최적화까지 전 과정 직접 수행했습니다.
2. 알고리즘 하이브리드화 능력: MCTS의 통계적 강점과 Minimax의 결정론적 정밀도를 상황별 트리거(빈칸 ≤ 40)로 결합한 메타-알고리즘 설계.
3. 실시간 추론 최적화: 100ms 제약 안에서 의미 있는 의사결정을 위해 비트보드, 록업테이블, SIMD, 트리 재활용 등 다층적 최적화 적용.
4. 객관적 검증 통과: 동일 룰 위에서 글로벌 10,007명과 직접 대결해 글로벌 Top 1.4%, 한국 1위 달성.

8.1 향후 확장 가능성

- 현재 무작위로 진행하는 MCTS를 신경망 기반으로 바꿔 AlphaZero 구조로 개선
- Minimax probe의 깊이를 시간 예산에 따라 동적으로 조정
- 정책-가치 분리 학습으로 시뮬레이션 단계 자체를 신경망으로 대체

9. 첨부

파일	내용
<code>source_ai_bot_ulimate_tictactoe/ultimate_tictactoe.cpp</code>	봇 전체 소스 (C++)